

The Implementation of LDFC in Vredrs 0.1.5

1 Introduction

Vredrs is a small virtual machine for a dynamically-typed language. Its garbage collector, LDFC (Lazy Deferred Free via Conservative scanning), implements the False-Root Aging Theory described in the companion paper. This paper describes the implementation of LDFC in Vredrs 0.1.5.

The collector is conservative: it scans the mutator’s stack and registers for values that resemble pointers, and treats each such value as a root. This avoids the bookkeeping that precise rooting demands, at the cost of *false roots*—values that are not pointers but happen to fall in the heap address range. The aging theory gives false roots a bounded lifetime: each false root is reclaimed after a small number of collection cycles, while true roots (reached by the heap trace) are preserved.

The implementation faces four engineering challenges that this paper addresses:

1. **Conservative scanning without compiler support.** Vredrs has no JIT and no stack maps. The collector must scan the native stack at each safepoint, using only the operating system’s signal-based context capture. We describe the scan routine and the heap-address filter that prunes non-heap values (sections 2 and 3).
2. **Concurrent marking with low pause time.** The collector runs a background marker thread alongside the mutator. Pause time is bounded by the safepoint work (root scan + barrier flush), not by heap size. We describe the epoch clock, the lazy-epoch reset that avoids an $O(N)$ mark-reset loop, and the incremental sweep that bounds per-safepoint work to $O(\text{batch})$ (sections 6 and 7).
3. **Write barrier cost.** The concurrent marker needs to know which pointer fields the mutator has modified. We describe the card-table write barrier that records stores with one byte write per store and no hash lookup (section 5).
4. **Generational extension.** A proof-of-concept tolerant generational collection (TGC) layers a young/old generation split on top of the aging theory, with a piggyback remembered set that adds one integer comparison to the write barrier (section 8).

The paper is organized as follows. Section 2 gives an architectural overview. Section 3 describes the core data structures (object header, object table, epoch clock). Section 4 covers allocation. Section 5 covers the write barrier. Section 6 covers marking (concurrent and synchronous). Section 7 covers sweeping and reclamation. Section 8 covers the generational extension. Section 9 reports measurements.

2 Architectural Overview

The collector is organized into six modules, totaling about 5,600 lines of Rust:

Module	Responsibility	Lines
<code>handle.rs</code>	Object header, <code>Gc<T></code> handle, allocation	658
<code>shared.rs</code>	Shared state (object table, root scanner, config)	311
<code>mutator.rs</code>	Per-thread mutator state, write barrier	271
<code>marker_local.rs</code>	Marker thread, mark cycle	437
<code>sweep.rs</code>	Sweeper, destruct queue, backpressure	414
<code>generational.rs</code>	TGC extension	456
<code>concurrent_table.rs</code>	Chunked object table	394
<code>roots.rs</code>	Conservative scan, DRD state machine	390
<code>epoch.rs</code>	Epoch clock, mark-state packing	217
<code>barrier.rs</code>	Global mark queue	246
<code>card_table.rs</code>	Card table	76
<code>runtime.rs</code>	Runtime facade	1306

2.1 Collection Cycle

A collection cycle has three phases, coordinated by a global epoch clock:

1. **Safepoint (STW).** The mutator pauses. The marker advances the epoch, scans roots (explicit + conservative), and drains the write-barrier queue. This phase is $O(\text{roots})$, independent of heap size.
2. **Marking (concurrent or STW).** The marker traces the object graph from the root set, coloring objects gray $\rightarrow$ black. In concurrent mode, this runs on a background thread while the mutator continues. In synchronous mode (used by tests), it runs to completion on the mutator thread.
3. **Sweeping (incremental, STW per batch).** The sweeper walks the object table in batches of 128 slots, reclaiming objects that are white and EBR-safe. Each batch is $O(\text{batch})$. The sweeper also runs the DRD (Defensive Re-verification Demotion) state machine that ages false roots.

The epoch clock is a single `AtomicU64`. Advancing it at safepoint makes every object’s prior mark state “stale” (effectively white) without an $O(N)$ reset loop—the lazy-epoch optimization described in section 3.

2.2 Concurrency Model

The collector supports two concurrency modes:

- **Synchronous** (single-threaded): `start_cycle` $\rightarrow$ `run_sync` $\rightarrow$ `sweep`. Used by tests and by the simple runtime.
- **Concurrent:** `start_cycle` $\rightarrow$ `run_background` (spawn thread) $\rightarrow$ mutator continues $\rightarrow$ `wait_for_completion` $\rightarrow$ `sweep`. The marker thread traces the graph while the mutator runs; the write barrier records concurrent modifications for the marker to pick up.

Multi-mutator mode spawns multiple `GcMutatorLocal` instances sharing one `Arc<GcShared>`. Each mutator has its own card table and allocation counter; the object table, root scanner, and sweeper are shared.

3 Data Structures

3.1 Object Header

Every GC object begins with a `GcBoxHeader`, followed by the object’s data. The header is `#[repr(C)]` so the data pointer is at a fixed offset (`sizeof::<GcBoxHeader>()`).

```
#[repr(C)]
pub struct GcBoxHeader {
    pub id: GcBoxId, // 8 bytes: slot index + 1
    pub last_touched_epoch: AtomicU64, // 8 bytes: EBR timestamp
    pub mark_state: AtomicU64, // 8 bytes: packed color + epoch + cons bit
    pub strong_count: AtomicU32, // 4 bytes: Gc<T> handle count
    pub fake_root_age: AtomicU8, // 1 byte: false-root age counter
    pub generation: AtomicU8, // 1 byte: 0=Young, 1=Old (TGC)
    pub survival_count: AtomicU8, // 1 byte: minor GC survivals (TGC)
    pub demotion_pending: AtomicU8, // 1 byte: DRD defer flag
    pub trace_fn: Option<unsafe fn>, // 8 bytes: child visitor
    pub drop_fn: Option<unsafe fn>, // 8 bytes: type-erased drop
}
```

The header is 56 bytes on 64-bit platforms. The `mark_state` field packs three pieces of information into one 64-bit atomic, which is the key to lock-free marking:

Bits	Field	Meaning
[0:2)	color	0=WHITE, 1=GRAY, 2=BLACK
[2]	conservative_root	1 if marked by conservative scan, 0 if by trace
[3:64)	mark_epoch	epoch in which the mark was set

Packing these into one atomic is essential: a separate `mark: AtomicU8 + mark_epoch: AtomicU64` would allow a torn read where the marker sees `color=GRAY` from cycle t and `epoch=t + 1` from a concurrent cycle, misjudging the object’s state. The 64-bit atomic guarantees atomicity of the (color, epoch, conservative) triple.

3.2 Lazy Epoch Reset

The `mark_epoch` field enables *lazy epoch reset*. In a conventional tri-color collector, each cycle begins by walking every object and resetting its color to white—an $O(N)$ operation. LDGC avoids this: at safepoint, the marker advances `GC_EPOCH` by one. An object is “stale” (effectively white) if its `mark_epoch < current_epoch`. The reset is $O(1)$.

The effective color of an object is computed as:

$$\text{effective_color}(o) = \begin{cases} \text{WHITE} & \text{if } \text{mark_epoch}(o) < \text{GC_EPOCH} \\ \text{stored_color}(o) & \text{otherwise} \end{cases}$$

This is read by `mark_color()` on every sweep decision. The cost is one comparison; the benefit is eliminating the $O(N)$ reset that would otherwise dominate pause time for large heaps.

3.3 Object Table

The object table is a chunked array of `AtomicPtr<GcBoxHeader>`, designed for concurrent push and concurrent read:

```
pub const CHUNK_SIZE: usize = 4096;

pub struct ConcurrentObjectTable {
    chunks: RwLock<Vec<Arc<[AtomicPtr<GcBoxHeader>; CHUNK_SIZE]>>>,
    len: AtomicUsize,
}
```

Push acquires a read lock on the chunk list (fast path), stores the pointer into the appropriate chunk, and returns the linear index. Only when the chunk list must grow does push upgrade to a write lock. This makes push contention-free in the steady state: multiple mutator threads can push concurrently, each writing to a different slot.

The object ID is the slot index plus one (`GcBoxId(idx + 1)`; zero is reserved for “invalid”). This means `lookup_header(id)` is a direct array index: `id.0 - 1`. There is no hash map, no mutex, and no allocation on the lookup path.

3.4 Epoch Clock

The epoch clock is a single global `AtomicU64`:

```
pub static GC_EPOCH: AtomicU64 = AtomicU64::new(0);
```

At each safepoint, the marker calls `GC_EPOCH.fetch_add(1, AcqRel)`. All objects whose `mark_epoch < new_epoch` become stale. The epoch is 61 bits wide (bits 3–63), which at 1 kHz cycle rate gives $\sim 10^{15}$ seconds (~ 290 million years) before wraparound—not a practical concern.

4 Allocation

Allocation is the mutator’s hottest path. The design minimizes per-allocation overhead to four operations:

1. `allocate_box_no_id(data)`: constructs a `Box<GcBox<T>>` on the Rust heap. The header is initialized with `mark_state = (current_epoch, WHITE)`, `strong_count = 1`, `last_touched_epoch = current_epoch`.
2. `shared.allocate_slot(header_ptr)`: pushes the header pointer into the object table, returning the slot index. This is one `fetch_add` (to get the index) and one `AtomicPtr::store` (to publish the pointer).
3. The ID is set to `GcBoxId(idx + 1)`.
4. `Gc::from_box(boxed)`: wraps the raw pointer in a `Gc<T>` handle.

```
pub fn allocate<T>(&self, data: T) -> Gc<T> {
    let mut boxed = allocate_box_no_id(data);
    let header_ptr: *mut GcBoxHeader = &mut boxed.header;
    let real_id = self.shared.allocate_slot(header_ptr);
    boxed.header.id = real_id;
    let alloc_size = std::mem::size_of::<T>()
        + std::mem::size_of::<GcBoxHeader>();
    self.bytes_allocated.set(
        self.bytes_allocated.get() + alloc_size);
    self.objects_allocated.set(
        self.objects_allocated.get() + 1);
}
```

```
Gc::from_box(boxed)
}
```

The `drop_fn` field is set at allocation time by a monomorphized `drop_impl::<T>`, which reconstructs `Box<GcBox<T>>` from the header pointer and drops it. This allows the destruct queue to free typed allocations without knowing `T` at runtime.

4.1 Allocation Cost

The per-allocation cost is:

- 1 `fetch_add` (object table index)
- 1 `AtomicPtr::store` (publish pointer)
- 2 `Cell::set` (allocation counters, no atomic)
- 1 `Box::new` (Rust allocator)

No mutex, no hash lookup, no allocation in the object table (chunks are pre-allocated). The measured allocation rate is 1.3 million objects per second on a single core (section 9).

5 Write Barrier

The concurrent marker needs to know which pointer fields the mutator has modified, so it can re-trace them. LDFC uses a card-table write barrier.

5.1 Card Table

The card table is a byte array, one byte per object-table slot:

```
pub struct CardTable {
    table: RefCell<Vec<u8>>,
}
```

A dirty byte (value 1) means “this object has been modified since the last safepoint; re-trace it.” The write barrier marks the *parent* dirty (not the child):

```
pub fn write_barrier(&self, _mark: u8,
                    parent_id: GcBoxId, child_id: GcBoxId) {
    if let Some(header) = self.shared.lookup_header(parent_id) {
        unsafe { (*header).touch(); } // refresh last_touched_epoch
    }
    let parent_idx = (parent_id.0 as usize).saturating_sub(1);
    self.card_table.mark_dirty(parent_idx);
}
```

Marking the parent (not the child) is a correctness decision. If the barrier marked the child, the child would become a stale root: the next cycle’s marker would mark it unconditionally, even if the parent had since dropped the reference. This would cause the “permanent exemption leak” where objects that were once live and then became garbage are never reclaimed. By marking the parent, the barrier ensures that the child is only re-traced if the parent is still reachable—exactly the conservative scanning invariant.

5.2 Barrier Cost

The write barrier costs:

- 1 `object_table.get(idx)` (RwLock read path, no mutex)
- 1 `AtomicU64::store(touch())`
- 1 byte write (card table)

No hash lookup, no `Vec::push`, no lock. The measured overhead is 9.7% on the allocation path (section 9).

5.3 Barrier Flush

At safepoint, the mutator flushes its card table into the global mark queue:

```
pub fn flush_card_table(&self) {
    let dirty = self.card_table.drain_dirty();
    if dirty.is_empty() { return; }
    let entries: Vec<GcBoxId> = dirty.iter()
        .filter_map(|&idx| {
            let ptr = self.shared.object_table.get(idx);
            if !ptr.is_null() { unsafe { Some((*ptr).id) } }
            else { None }
        }).collect();
    if !entries.is_empty() {
        self.shared.global_queue.push_batch(entries);
    }
}
```

The global mark queue is a `crossbeam_queue::SegQueue`, which is lock-free and multi-producer. Multiple mutators can flush concurrently without contention.

5.4 Stale-Root Avoidance

The global mark queue is consumed by the marker during `run_sync` (or the background thread). A critical correctness property: barrier entries are *not* treated as unconditional roots. The marker only re-traces a barrier entry if it has already been marked gray or black in the current cycle:

```
while let Some(id) = shared.global_queue.pop() {
    if let Some(h) = shared.lookup_header(id) {
        let state = h.mark_state.load(Acquire);
        let cur = GC_EPOCH.load(Acquire);
        let (epoch, color) = unpack(state);
        if epoch == cur && (color == GRAY || color == BLACK) {
            // parent is reachable; re-trace to find new children
            if let Some(trace_fn) = h.trace_fn {
                unsafe { trace_fn(h, &mut visitor); }
            }
        }
        // parent not marked -> dead -> skip (no stale root)
    }
}
```

This prevents the stale-root leak: a dead parent (not reached by the trace) has `mark_epoch < current`, so the barrier entry is skipped. Its children are not re-marked, and they are reclaimed normally.

6 Marking

6.1 Mark Cycle

The mark cycle begins at safepoint. `start_cycle` does four things:

1. Advances `GC_EPOCH` by one (lazy reset).
2. Marks explicit roots (registered via `register_root`) gray.
3. Performs conservative scan: scans the mutator's stack and register context for values that fall in the heap address range, resolves each to an object ID, and marks it gray with the conservative bit set.
4. (The barrier queue is not drained here; it is consumed during `run_sync` as described in section 5.)

6.2 Conservative Scan

The conservative scan walks the mutator's stack from the current stack pointer upward, reading pointer-sized values:

```
pub unsafe fn conservative_scan<F>(  
    &self, heap_start: usize, heap_end: usize,  
    header_lookup: &dyn Fn(usize) -> Option<GcBoxId>,  
    mut visitor: F,  
) where F: FnMut(GcBoxId) {  
    let mut local = 0usize;  
    let stack_ptr = &mut local as *mut usize as usize;  
    const SCAN_WINDOW: usize = 256 * 1024;  
    const RED_ZONE_SIZE: usize = 128;  
    let red_zone_start = stack_ptr.saturating_sub(RED_ZONE_SIZE);  
    let scan_end = stack_ptr.saturating_add(SCAN_WINDOW);  
  
    // scan red zone [SP-128, SP) for x86-64 leaf-frame locals  
    let mut addr = red_zone_start;  
    while addr < stack_ptr {  
        let value = *(addr as *const usize);  
        if value >= heap_start && value < heap_end {  
            if let Some(id) = header_lookup(value) {  
                visitor(id);  
            }  
        }  
        addr += std::mem::size_of::<usize>();  
    }  
    // scan main stack [SP, SP+256KB)  
    let mut addr = stack_ptr;  
    while addr < scan_end {  
        let value = *(addr as *const usize);  
        if value >= heap_start && value < heap_end {  
            if let Some(id) = header_lookup(value) {  
                visitor(id);  
            }  
        }  
        addr += std::mem::size_of::<usize>();  
    }  
}
```

The scan covers the x86-64 red zone (128 bytes below SP, used by leaf functions without frame setup) and a 256 KB window above SP (covering typical call depths). Each value is filtered by the heap address range `[heap_min, heap_max]`—a range maintained $O(1)$ by `fetch_min/fetch_max` on every allocation. Values outside the range are skipped without a hash lookup.

For values inside the range, `header_lookup` resolves the pointer to a `GcBoxId` via the `ptr_to_id_cache`, a `HashMap` built lazily at safepoint. This filters out non-GC heap pointers (e.g., from Rust’s own allocator)—only `GcBox` headers resolve to IDs.

6.3 Mark State Transitions

The marker uses two CAS operations:

White → **Gray** (`try_mark_gray`): CAS from stale or (current, WHITE) to (current, GRAY). If the object was already marked this cycle, the CAS fails and the object is not re-pushed. This prevents cycles from overflowing the mark stack.

Gray → **Black** (`mark_black`): after all children are traced, the object is colored black. This uses `fetch_update` to preserve the conservative bit: a conservative-marked object (bit set) transitioning to black keeps the bit, so the sweeper knows it was not trace-confirmed.

6.4 Concurrent Marking

In concurrent mode, `run_background` spawns a thread that consumes the mark stack:

```
pub fn run_background(&mut self) {
    let shared = self.shared.clone();
    let mark_stack = MarkStackSend(
        std::mem::take(&mut self.mark_stack));
    self.bg_thread = Some(std::thread::spawn(move || {
        let mut stack = mark_stack;
        while let Some(obj) = stack.0.pop() {
            let header = unsafe { &*obj };
            if let Some(trace_fn) = header.trace_fn {
                let mut visitor = |child_id: GcBoxId| {
                    if let Some(h) = shared.lookup_header(child_id) {
                        if unsafe { (*h).try_mark_gray() } {
                            stack.0.push(h);
                        }
                    }
                };
                unsafe { trace_fn(obj, &mut visitor); }
            }
            header.mark_black();
        }
        stack.0.len()
    }));
}
```

The mutator continues running during marking. When the mutator modifies a pointer field, the write barrier records the parent in the card table; the marker picks it up on the next barrier-queue drain. `wait_for_completion` joins the background thread, after which sweeping can begin.

7 Sweeping and Reclamation

7.1 Incremental Sweep

Sweeping is incremental: each safepoint advances the sweep cursor by a fixed batch (128 slots). This bounds the per-safepoint work to $O(\text{batch})$, independent of heap size.

```
pub unsafe fn sweep_range(&mut self, slots: &[Slot], _base: usize) {
    let current_epoch = GC_EPOCH.load(Acquire);
    for slot in slots {
        let obj = slot.get();
        if obj.is_null() { continue; }
        let header = &*obj;
        let action = RootScanner::drd_process(header);
        match action {
            DrdAction::Keep | PendingRescued | PendingHeld => {
                self.inc_kept();
            }
            DrdAction::AgeAdvanced => {
                if is_epoch_elapsed(
                    header.last_touched_epoch.load(Acquire),
                    current_epoch)
                {
                    if let Some(obj) = slot.take_if_reclaimable() {
                        self.retire_object(obj);
                    }
                }
            }
            DrdAction::PendingSet => { self.inc_false_roots_aged(); }
            DrdAction::Demoted => {
                if let Some(obj) = slot.take_if_reclaimable() {
                    self.retire_object(obj);
                }
            }
        }
    }
}
```

7.2 CAS Take-Away

Reclamation uses a CAS take-away to guarantee unique ownership. `take_if_reclaimable` atomically nulls the slot:

```
pub unsafe fn take_if_reclaimable(&self, idx: usize)
    -> Option<*mut GcBoxHeader>
{
    let chunk_idx = idx / CHUNK_SIZE;
    let slot_idx = idx % CHUNK_SIZE;
    let chunks = self.chunks.read().unwrap();
    let chunk = chunks.get(chunk_idx)?;
    let obj = chunk[slot_idx].load(Acquire);
    if obj.is_null() { return None; }
    if (*obj).is_reclaimable() {
        match chunk[slot_idx].compare_exchange(
            obj, std::ptr::null_mut(), AcqRel, Acquire
        ) {
            Ok(_) => Some(obj),
            Err(_) => None,
        }
    }
}
```

```

    }
  } else { None }
}

```

Only one thread can win the CAS; the winner gets unique ownership and can safely call `drop_fn`. This prevents double-free even under concurrent sweeping.

7.3 EBR Delay

An object is reclaimable only if:

1. `strong_count == 0` (no live handles)
2. `mark_epoch < current_epoch` (not marked this cycle = unreachable)
3. `is_epoch_elapsed(last_touched_epoch, current_epoch)`: at least 2 epochs have elapsed since the last mutator touch.

The third condition is the epoch-based reclamation (EBR) delay. It ensures that any mutator thread that was active when the object was last touched has exited the mutator section before the object is freed. The delay is 2 epochs (1 to advance past the touch epoch, 1 to ensure the mutator has reached a safepoint).

7.4 Destruct Queue

Reclaimed objects are not freed immediately; they are pushed to a destruct queue. At end of cycle, `process_destruct_queue` calls `drop_fn` on each, which reconstructs `Box<GcBox<T>>` and drops it, invoking `T`'s `Drop` and freeing the heap allocation.

The destruct queue has backpressure: when it exceeds 4096 entries, allocation blocks until it drains below 2048. This prevents unbounded destruct-queue growth under heavy reclamation.

7.5 DRD State Machine

The sweep runs the DRD (Defensive Re-verification Demotion) state machine on each object, which implements the two-safepoint consensus for false-root reclamation:

strong	mark	pending	cons	Action
> 0	any	any	any	Keep
0	BLACK/GRAY	0	any	Keep
0	BLACK/GRAY	1	1	PendingHeld (keep pending)
0	BLACK/GRAY	1	0	PendingRescued (clear pending, reset age)
0	WHITE	1	—	Demoted (<code>force_demote + reclaim</code>)
0	WHITE	0	—	<code>age++</code> ; if > MAX: PendingSet, else AgeAdvanced

The `PendingRescued` action is the DRD rescue: if an object was pending (age exceeded threshold) but the trace reaches it this cycle (conservative bit cleared), the prior white cycle was a transient scan miss, and the object is rescued.

8 Generational Extension

The TGC (Tolerant Generational Collection) extension layers a young/old generation split on top of the aging theory. It is a proof of concept; the goal is to show that the aging theory composes with generational collection without requiring a separate write barrier.

8.1 Generation Tag

Each object header carries a 1-byte generation tag: 0 = Young, 1 = Old. Objects are allocated Young and promoted to Old after surviving 2 minor collections. The tag is read by the write barrier to detect cross-generation references.

8.2 Piggyback Remembered Set

The TGC write barrier piggybacks on the existing card-table barrier:

```
pub fn write_barrier(&self, mark: u8,
                    parent_id: GcBoxId, child_id: GcBoxId) {
    self.mutator.write_barrier(mark, parent_id, child_id);
    // TGC extension: check cross-generation reference
    if let Some(ph) = self.shared.lookup_header(parent_id) {
        if let Some(ch) = self.shared.lookup_header(child_id) {
            unsafe {
                let pgen = (*ph).generation.load(Acquire);
                let cgen = (*ch).generation.load(Acquire);
                if pgen == GEN_OLD && cgen == GEN_YOUNG {
                    self.remembered_set.insert(child_id);
                }
            }
        }
    }
}
```

The cost is one extra `lookup_header` + two `AtomicU8::load` per store. The remembered set is a fixed-capacity (256-entry) ring buffer.

8.3 Minor GC

Minor GC scans only the young generation (via the `YoungIndex`, a chunked list of young object indices) plus the remembered set. It does not scan the old generation. This makes minor GC $O(|\text{Young}| + |\text{RS}|)$, independent of old-generation size.

Minor GC is triggered by young-generation byte accumulation (1 MB threshold), not by allocation count. This reduces GC frequency for allocation-heavy workloads.

8.4 Young Index: Chunked Design

The `YoungIndex` uses a chunked design to avoid locking on every allocation:

```
pub struct YoungIndex {
    chunks: Mutex<Vec<Vec<usize>>>, // full chunks
    current: Mutex<Vec<usize>>, // current (mutable) chunk
    len: AtomicUsize,
}
const YOUNG_CHUNK_SIZE: usize = 1024;
```

`add` pushes to the current chunk and only locks the chunk list when the current chunk is full (every 1024 allocations). This reduces lock contention by a factor of 1024 compared to a naive `Mutex<Vec>`.

8.5 Hybrid Lease

The TGC collector also carries an extrinsic lease table (`ExtrinsicLeaseTable`), which closes the persistent-false-root boundary. The intrinsic age alone cannot bound a $P = 1$ persistent false root (age freezes at 0). The extrinsic lease expires after $\ell = 5$ epochs regardless of re-marking, so the persistent false root becomes white once the lease expires and is reclaimed by the EBR delay.

```
pub struct ExtrinsicLeaseTable {
    entries: DashMap<usize, ExtrinsicLeaseEntry>,
    lease_term: u64,
}

impl ExtrinsicLeaseTable {
    pub fn anchor_or_check(&self, addr: usize,
                           current_epoch: u64) -> bool {
        if let Some(mut entry) = self.entries.get_mut(&addr) {
            if entry.trace_exempt { return false; }
            let elapsed = current_epoch
                .wrapping_sub(entry.first_seen_epoch);
            elapsed <= self.lease_term
        } else {
            self.entries.insert(addr, ExtrinsicLeaseEntry {
                first_seen_epoch: current_epoch,
                trace_exempt: false,
            });
            true
        }
    }
}
```

The hybrid bound is $L \leq \ell + 2$ for persistent false roots, while transient and oscillating false roots retain their intrinsic bounds ($\theta + 3$ and $P(\theta + 1) + 2$).

9 Measurements

All measurements are on a single core, single mutator thread, to isolate GC behavior from scheduling noise.

9.1 Allocation Rate

Mode	Allocations per second
No barrier	1,258,865
With barrier	1,164,375

The write barrier costs 9.7% on the allocation path.

9.2 GC Overhead

Mode	Allocs/s	Peak live	Total time
No GC	1,304,679	50,000	38.3 ms
With GC (incremental)	904,166	2,000	55.3 ms

The with-GC variant has a $1.4\times$ slowdown relative to no-GC. The peak live memory is bounded at 2,000 (the EBR delay window), versus 50,000 for no-GC.

9.3 Pause Time

Heap size	p50 (ns)	p95 (ns)	max (ns)
1,000	89,129	389,834	389,834
10,000	8,802	13,012	14,507
50,000	9,721	13,716	17,462
500,000	9,721	13,716	17,462

For heaps from 10,000 to 500,000 objects (a $50\times$ range), the p50 pause stays between 8.8 and $9.7\mu\text{s}$. The pause is $O(\text{batch})$ and independent of heap size.

9.4 TGC Throughput

Configuration	Allocs/s
LDFC (no generations)	443,881
TGC (with generations)	944,508

TGC gives a $2.1\times$ throughput improvement over LDFC for the cross-generation workload.

9.5 Long-Run No-Leak

A 20-round workload (100 temporary objects per round, each traced then made garbage) shows:

- Live objects stable at 101 (root + current round’s temporaries)—no monotonic growth.
- 1,900 Drop invocations (19 rounds $\times$ 100), confirming actual memory reclamation.

The `object_table` index space grows (reclaimed slots are not reused without compaction), but this is an index-space issue, not a memory leak—the actual object memory is freed by `drop_fn`.

10 Conclusion

The LDFC implementation in Vredrs 0.1.5 demonstrates that the False-Root Aging Theory is implementable in a real virtual machine with modest engineering effort. The key implementation decisions—lazy epoch reset, card-table write barrier marking the parent, CAS take-away reclamation, and the DRD state machine—together achieve:

- Pause time independent of heap size (8.8– $9.7\mu\text{s}$ p50 for 10k–500k objects).
- Write barrier overhead of 9.7%.

- GC slowdown of $1.4\times$ relative to no-GC.
- No long-run memory leak (1,900 objects freed over 20 rounds).
- Generational extension with $2.1\times$ throughput improvement.

The implementation is about 5,600 lines of Rust, organized into 13 modules. It runs on standard Unix platforms using signal-based context capture for conservative scanning, and requires no compiler cooperation beyond safepoints.